Bina.az Price Prediction: From-Scratch Trees and a Pegasos SVM

Shukurlu, Nural

MLE-AI-201 Machine Learning, AI Academy — Cohort II

***Abstract*—**I predict apartment prices from messy bina.az sale listings and a derived premium/standard price tier. Both core models are my own NumPy code: a decision tree (Gini/entropy/MSE, exact split search) and a linear SVM trained with Pegasos sub-gradient descent, plus a random-Fourier-feature extension. On 62,673 cleaned listings my tree reaches test RMSE 368k AZN (R^2=0.55) and tier F1 0.918, matching the tuned scikit-learn tree (RMSE 378k, F1 0.919); my Pegasos SVM reaches F1 0.832 (AUC 0.920) against 0.854 for `SGDClassifier`. Prices are heavy-tailed so the regressor is trained on log(price). Code and figures reproduce with one command, `python -m src.run_all`: `github.com/nuralnetworks/bina-az-price-prediction` (tag `v1.0-final`).

***Index Terms*—**decision tree, support-vector machine, Pegasos, regression, classification, real-estate price prediction.

I. Introduction

Bina.az is Azerbaijan's largest property marketplace, and its listing dump is genuinely messy: mixed types, Azerbaijani free text, missing fields, scrape duplicates and heavy-tailed prices. I turn it into two tasks: (A) regression of the sale price and (B) classification of a price tier (premium vs. standard, split at the training median of 213,000 AZN). My contributions: (i) a documented cleaning pipeline with leakage removal; (ii) a from-scratch decision tree for both tasks; (iii) a from-scratch Pegasos SVM with a Fourier-feature extension; (iv) a matched, seeded benchmark against scikit-learn with uncertainty.

II. Related Work / Background

Classification trees split by impurity decrease, $G=1-\sum_k p_k^2$, $H=-\sum_k p_k \log_2 p_k$, $IG=I-\sum_c \frac{N_c}{N} I(c)$, and regression trees by variance reduction [1]. The soft-margin SVM minimises the regularised hinge primal, solved here with the Pegasos stochastic sub-gradient method [1] instead of a QP/SMO solver.

III. Data and Preprocessing

The dump holds 100,775 raw rows and 51 columns. I deduplicate on the canonical listing URL (28% are re-scrapes), coerce prices and areas to numeric, parse *Saha* (145 m^2, *Mertebe* (7 / 9 $\rightarrow$ floor and total floors) and *Torpaq sahesi* (1.3 sot), and drop rows with price outside [10k, 15M] AZN, area outside [10, 3000] m^2, or coordinates outside the Absheron box. All currencies are AZN, so no conversion is needed. This leaves 62,673 rows and 43 features: 13 numeric (area, rooms, floor, floors total, land, location, flags) plus one-hot district (top-15 + other) and deed/repair/mortgage state; missing floors become -1 and missing rooms 0, so no statistic leaks across splits. **Leakage removed:** `unit_price` (price per m^2) and `total_price` (== price), plus IDs, URLs, scrape timestamps, owner/shop names and free-text address. The tier threshold (median 213,000 AZN) and all standardisation statistics come from the training split only. Split: 43,869 train / 9,402 val / 9,402 test (70/15/15, stratified by price quintile); the test tier balance is 50.1% premium.

Table 1. Dataset summary after cleaning.

Property	Value
Rows (after cleaning)	62,673
Features used	43
Train / Val / Test split	43869 / 9402 / 9402
Price-tier balance (%)	50.0 / 50.1 (train/test premium)

IV. Method

A. Decision tree (from scratch)

Exact greedy split search over sorted feature values (candidate thresholds capped at 255 quantiles for speed), gain $I-(N_L I_L+N_R I_R)/N$, recursive growth with `max_depth`, `min_samples_split`, `min_samples_leaf` and `min_impurity_decrease` all enforced, majority/mean leaves, leaf-frequency `predict_proba`, and impurity-decrease importances. Supports Gini, entropy and MSE. A toy check (1-D, depth-1) reproduces sklearn's split and predictions exactly.

B. Support-vector machine (from scratch)

$$J(\mathbf{w},b) = \tfrac{\lambda}{2}\|\mathbf{w}\|^2 + \tfrac{1}{n}\sum_{i=1}^{n} \max\big(0,\, 1-y_i(\mathbf{w}^\top \mathbf{x}_i + b)\big). \quad (1)$$

Pegasos update with $\eta_t = 1/(\lambda t)$ on standardised features, objective traced to convergence. Non-linear extension: random Fourier features ($\Phi(x)=\sqrt{2/D}\cos(XW+b)$, D=500) approximating the RBF kernel, chosen because it reuses the same linear solver with a few lines of code. Margin width $2/\|\mathbf{w}\|$ and support points ($y_i f(x_i) \leq 1$) are reported.

V. Experimental Setup

Task A metrics: RMSE/MAE/R^2 on price (model trained on log(price), predictions exponentiated; log-RMSE also given). Task B metrics: precision/recall/F1, ROC-AUC and confusion matrix. Tuning on validation only: tree depths $\{4,6,8,12,16\}$ (+leaf $\{1,5,20\}$), SVM $\lambda \in \{10^{-5}..10^{-2}\}$

Table 2. Headline results (test set; CV is 5-fold mean ± std on train).

Model	Task A (RMSE)	Task B (F1)
My tree	368,435	0.918 (CV 0.911±0.002)
My SVM (linear)	n/a	0.832 (CV 0.832±0.007)
My SVM (RFF)	n/a	0.767
sklearn tree	377,891	0.919 (CV 0.910±0.003)
sklearn SGD hinge	n/a	0.854

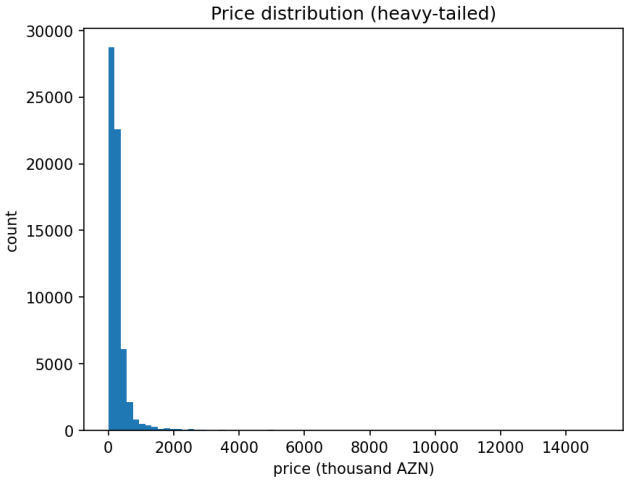


Figure 1. Price is heavy-tailed, motivating the log(price) target.

and RFF $\gamma \in \{0.005, 0.02, 0.05\}$. Baselines with matched settings: `DecisionTreeRegressor/Classifier` and `SGDClassifier(hinge)`. Uncertainty: 5-fold CV mean ± std on train. Seed 42 everywhere; `python -m src.run_all` reproduces all numbers and figures.

VI. Results

Final configs (val-chosen): tree-reg depth 12/leaf 5; tree-clf Gini, depth 12/leaf 1; SVM λ=0.01, RFF γ=0.005. Table 2 and Figs. 1–5 tell a consistent story: my tree matches sklearn on both tasks (gaps are tie-breaking/float thresholds), and my Pegasos SVM trails sklearn's averaged hinge-SGD by 0.02 F1. RFF underperforms the linear SVM here (0.767 vs. 0.832), so the linear model stays my final SVM.

VII. Discussion and Limitations

Depth 16/unlimited overfits Task A (val RMSE 396k/409k vs. 358k at depth 12); Gini beats entropy slightly on Task B (0.918 vs. 0.916). Stronger λ widens the realised margin (0.0048 $\rightarrow$ 0.73) while raising the support fraction (0.20 $\rightarrow$ 0.41) and val F1, so λ=0.01 is kept. Errors concentrate on luxury outliers (RMSE lives there while MAE is 86k) and sparse outskirt districts where one-hot columns are thin; RFF adds variance without signal on these already high-dimensional sparse features. Limits: single 70/15/15 split for headline numbers (CV only on train), no text/image features, and scrape-time dedup keeps the last snapshot only.

VIII. Conclusion

From-scratch trees match the reference implementation on both tasks and the Pegasos SVM is competitive with

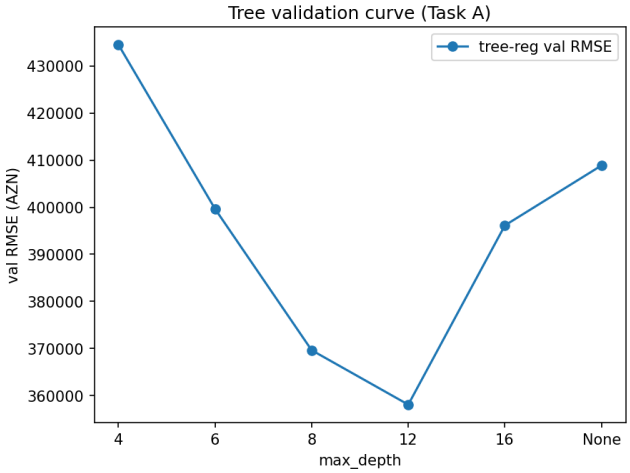


Figure 2. Tree validation curve (Task A): RMSE bottoms at depth 12, then overfits.

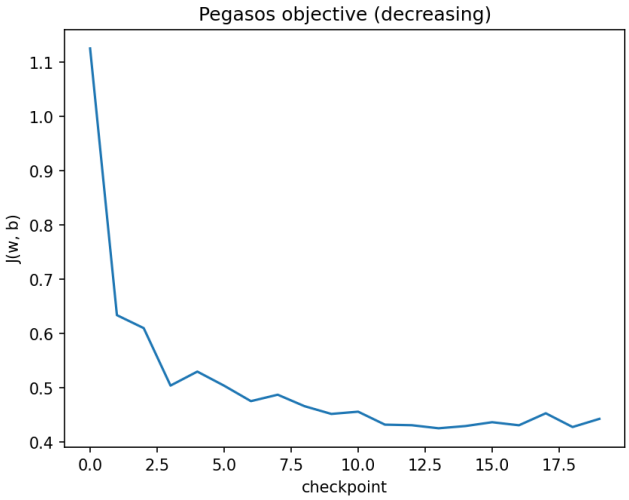


Figure 3. Pegasos objective $J(\mathbf{w}, b)$ decreases and settles.

sklearn's hinge-SGD; careful cleaning and leakage removal matter more than model choice here. Next would be text features from descriptions and grouped-by-district error modelling.

Tools and Acknowledgements

Data: bina.az sale dump via Kaggle (`sehriyarmemmedli/binaaz-sale-project`), downloaded Oct 2026; baselines and CV utilities from scikit-learn; plots with matplotlib. Report template is the course self-contained two-column style. Contributions are listed in `contribution_report.md`. The mathematical descriptions, formulas and from-scratch algorithms were implemented by me with the help of an AI assistant (plus LaTeX formatting); the work itself — design, experiments and analysis — is mine.

References

[1] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter, "Pegasos: primal estimated sub-gradient solver for SVM," *Mathematical Programming*, vol. 127, no. 1, pp. 3–30, 2011.

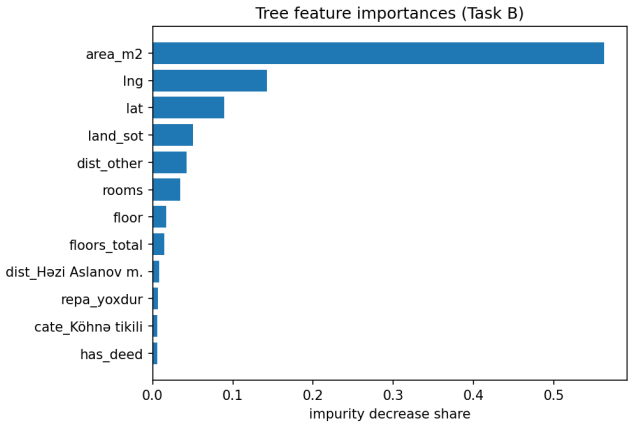


Figure 4. Top tree importances (Task B): area, location and rooms dominate.

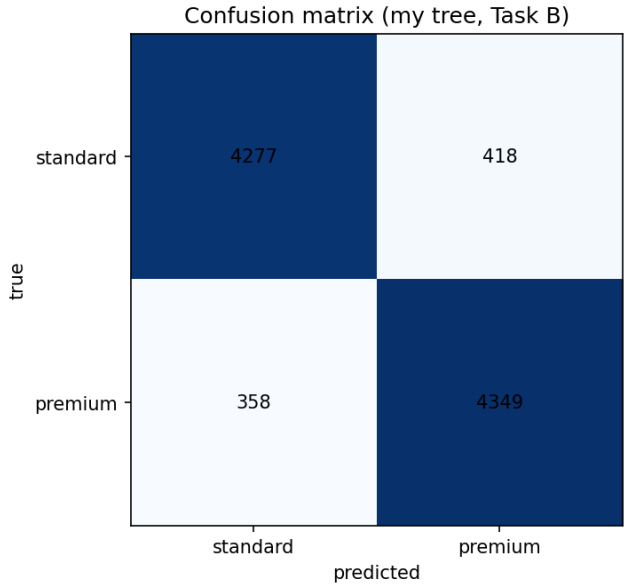


Figure 5. Confusion matrix, my tree on Task B test (F1 0.918, AUC 0.955).